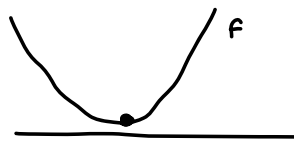


lecture 9 : Minimization and analysis



$$\Leftrightarrow Ax = b$$

Chief problem(s): minimization of multivariable functions $\xrightarrow{\text{Problem 1}}$ multiple dimensions possibly millions

$\xrightarrow{\text{Problem 2}}$ Does this exist?
Is it unique?

Goal for NN: find parameters s.t NN makes good predictions

Specially in neural networks (NN)
many, many parameters

we measure this using a loss function

e.g.
$$L(\theta) = \frac{1}{N} \sum_{i=1}^N (\underbrace{\phi_\theta(x_i)}_{\text{neural network data}} - y_i)^2$$
 Mean squared error

Good predictions iff loss is minimal

Minimization problem
find parameter θ^* s.t
 $\theta^* = \arg \min_{\theta} L(\theta)$

How can we find these parameters? θ^* .

- * Millions of parameters
- * Complicated $L(\theta)$ geometry and lack of nice properties

our only chance is to use numerical methods.

In general, we have to minimize $\phi: \mathbb{R}^n \rightarrow \mathbb{R}$ where:

$$\phi(s) = L(f(s, \underbrace{w_{in}}_{\text{parameters}}, \underbrace{w_{out}}_{\text{given output}})$$

$\nwarrow$ $\nearrow$
NN given input

$\nwarrow$ loss function $\nearrow$ will depend on the structure of the network.
 $\uparrow$ measures distance between output of NN: $f(s, w_{in})$ and the real output w_{out} .

Example: Suppose we have a sample of 10^4 images of size (480×460) each:

$$w_{in} = [w_1, \dots, w_{10^4}] \quad \text{s.t} \quad w_i \in \mathbb{R}^{480 \times 460}$$

We will classify these images in 10 different groups.

$$\Rightarrow W_{out} = [W_1, \dots, W_{10^4}] \quad \text{s.t.} \quad W_i \in \mathbb{R}^{10} \quad (\text{e.g. one-hot encoding}).$$

Suppose our neural network does the following:

$$W_j \xrightarrow{A_1 x + b_1} A_1 W_j + b_1 \xrightarrow{A_2 x + b_2} A_2 (A_1 W_j + b_1) + b_2$$

Parameters: $A_1 \in \mathbb{R}^{100 \times (480 \cdot 460)}$, $A_2 \in \mathbb{R}^{10 \times 100}$

$W_j \in \mathbb{R}^{480 \cdot 460}$ (flattened image)

$A_1 W_j + b_1 \in \mathbb{R}^{100}$ (Hidden layer with 100 neurons)

$A_2 (A_1 W_j + b_1) + b_2 \in \mathbb{R}^{10}$ (= W_j predicted output)

Our loss function would have the shape

$$\phi(s) = \sum_{j=1}^{10^4} \|A_2 (A_1 w_j) + b_1 - W_j\|^2$$

How can we minimize this?

Remark: Notice that if $\phi(x) = \frac{1}{2} Ax \cdot x - bx \Rightarrow \phi'(x) = Ax - b$,

hence minimizing ϕ (and thus finding x s.t. $\phi'(x) = 0$) solves $\underbrace{Ax - b = 0}_{Ax = b!}$

Minimization theory

Let's assume $\phi: \mathbb{R}^n \rightarrow \mathbb{R}$ is differentiable and it has a minimum at s_0 . Then

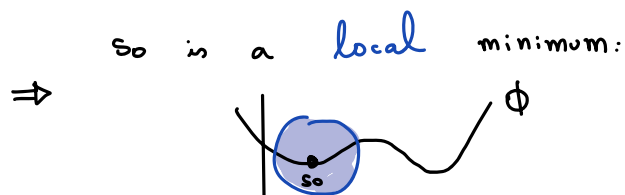
$$\nabla \phi(s_0) = 0 \Rightarrow \begin{cases} \partial_1 \phi(s_0) = 0 \\ \partial_2 \phi(s_0) = 0 \\ \vdots \\ \partial_n \phi(s_0) = 0 \end{cases}$$

If we know $\partial_i \phi$ exactly, this leads to a system of N equations with N unknowns.

Problems: $\begin{cases} - \text{usually non-linear system} \\ - \text{Is this } s_0 \text{ unique? does it exist? (we need extra assumptions)} \\ - \text{What if } s_0 \text{ is actually a maximum?} \end{cases}$

Extra assumptions:

- $\phi(s_0)$ twice differentiable
- $\phi''(s_0) \in \mathbb{R}^{n \times n}$ has only positive eigenvalues



- If ϕ is continuous everywhere and we are looking for parameters $s \in [-M, M]^n$ (compact space)

Then, we can ensure ϕ attains a minimum/maximum in $[-M, M]^n$.

How can we solve $\nabla\phi(s_0) = 0$?

Idea 1: Newton type methods

- ↳ Problems: - We need to compute the derivative of $\nabla\phi$, i.e. $\phi'' \in \mathbb{R}^{n \times n}$ which means we have to compute N^2 derivatives.
- Newton convergence is only local. But if we are close enough, convergence rapidly speeds up.

Is there a more efficient, simpler method?

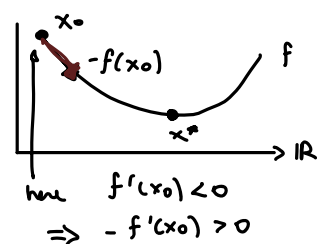
Consider the following example for visualization:



Here $\phi: \mathbb{R}^2 \rightarrow \mathbb{R}$ our loss function. To reach the minimum, we need we should aim for the downmost point of the surface.

Idea 2: Start with an initial guess, then take steps of certain length in the most downwards direction, while staying in the surface of ϕ .

↳ most efficient direction possible:



Generalizing for multiple dimensions:

Steepest descent direction: $-\nabla\phi(s)$

How long should this direction be?

Lecture 10: The gradient algorithm

Given a function $\phi: \mathbb{R}^n \rightarrow \mathbb{R}$, our goal is to find the minimum of it through iteration: we start with an initial guess s^0 and then:

$$s^{n+1} = \underbrace{s^n - \alpha \nabla \phi(s^n)}$$

At each iterate, we choose the steepest direction, and move in that direction with a given step-size α .

Stopping criterion:

① $|\nabla \phi(s^n)| < \text{tolerance}$.

That is, if the gradient is very small, i.e., we have reached a flat region, we stop.

② $|s^{n+1} - s^n| < \text{tolerance}$.

If the length between steps is small, we stop.

Assumptions for convergence of the gradient method

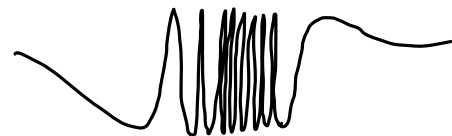
1. $\nabla \phi$ should be Lipschitz continuous, i.e., $|\nabla \phi(s_j) - \nabla \phi(s_k)| \leq L |s_j - s_k|$

for some $L > 0$. This ensures that $\nabla \phi$ does not "explode" or change too abruptly at some point:

Lipschitz:



not Lipschitz: e.g. $\sin(1/x)$



2. ϕ should be convex.

By 1 and 2, and choosing a learning rate $\alpha \leq \frac{1}{L}$ our iteration satisfies

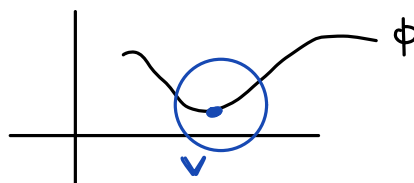
$$\phi(s^n) - \min \phi \leq 2L \underbrace{|s^0 - s^{\text{opt}}|}_n^2$$

where $s^{\text{opt}} \in \mathbb{R}^n$ is our set of parameters that minimizes ϕ , i.e. $\phi(s^{\text{opt}}) = \min \phi$

This tells us that exact error in the gradient method is proportional to $\boxed{1/n}$ at every step of the iteration.

Remark: Convexity is a very strong assumption, and, in real life situations, we can't know for sure ϕ is convex.

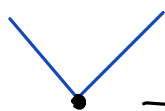
we can relax this assumption to local convexity: ϕ is convex in a neighborhood V of where the minimum is.



Learning rate α should be chosen small enough to stay inside V .

Differentiability of $\nabla\phi$ is important to avoid problems like the following:

Suppose $\phi(x) = \max\{f(x), g(x)\}$ where f, g are differentiable. The point x^* where the maximum changes (say e.g. from f to g) is sudden, which leads to sharp corners where $\nabla\phi$ does not exist.

 $\rightarrow$ at $x=x^*$, automatic differentiation will fail

In practice, how do we choose the stepsize α ?

Theoretically, $\alpha \leq \frac{1}{L}$, where $L = \max_x |\phi''(x)|$ (if ϕ'' is twice diff.) which is not realistic for larger problems.

$\Rightarrow$ **Adaptive learning rate:** at every step in the iteration test:

$$|\nabla\phi(s_j) - \nabla\phi(s_k)| < L|s_j - s_k|$$

If α fails, pick another one and repeat.

Problem: • If α is too small, very slow convergence

• If α is too large, we overshoot and miss the minimum.

Remark: We should always normalize our data! For example, consider

$$\phi \quad \text{and} \quad \phi^* = 10\phi \quad (\text{e.g. changing to different units})$$

Then the gradient method for these two functions is

$$s^{n+1} = s^n - \alpha \nabla \phi \quad (1)$$

and

$$s^{n+1} = s^n - 10\alpha \nabla \phi \quad (2)$$

respectively. Notice that both functions ϕ and ϕ^* attain the minimum at the same place, but the learning rate in (2) is much higher and thus it can lead to overshoot.

Example: (2 dimensions):

Task: Find (x^*, y^*) that minimizes $\phi(x, y) = 2x^2 - 4xy + y^4$.

1. We compute the gradient of ϕ :

$$\nabla \phi(x, y) = \begin{pmatrix} 4x - 4y \\ -4x + 4y^3 \end{pmatrix}$$

2. We choose step-size α : $\alpha = 0.2$.

3. Choose starting point (important!)

$$\hookrightarrow \text{if } s^0 = (0, 0) \Rightarrow \nabla \phi(0, 0) = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$$

at iteration stagnates: $s^1 = s^2 = \dots$

but $(0, 0)$ is not a minimum.

Choose $s^0 = (0.1, 0.1)$ 

(random choice is also a possibility).

4. Iterate: $s^1 = s^0 - \alpha \nabla \phi(s^0)$

$$= \begin{pmatrix} 0.1 \\ 0.1 \end{pmatrix} - 0.2 \begin{pmatrix} 0 \\ -0.036 \end{pmatrix} = \begin{pmatrix} 0.1 \\ 0.107 \end{pmatrix}$$

$$\vdots$$

$$s^8 = \begin{pmatrix} 0.873 \\ 0.945 \end{pmatrix}, \quad s^9 = \begin{pmatrix} 1.001 \\ 0.982 \end{pmatrix}, \quad s^{10} = \begin{pmatrix} 1.141 \\ 0.920 \end{pmatrix}$$

$\alpha = 0.2$ was too large and we skipped minimum site (1)

5. Choose smaller α learning rate: $\alpha = 0.1$:

$$s^8 = \begin{pmatrix} 0.386 \\ 0.567 \end{pmatrix}, \dots, s^{13} = \begin{pmatrix} 0.742 \\ 0.874 \end{pmatrix}, \dots, s^{19} = \begin{pmatrix} 0.941 \\ 0.975 \end{pmatrix}$$

(even in a simple case like this, convergence is slow).

there is another problem: if instead we start at $s^0 = \begin{pmatrix} -0.1 \\ -0.1 \end{pmatrix}$, with $\alpha = 0.1$.

the gradient method approaches $\begin{pmatrix} -1 \\ -1 \end{pmatrix}$, the other local minimum!

Graph of ϕ has two "valleys"



There is no general rule for choosing s^0 , only educated guesses (say e.g. if the parameters of the network have a real meaning).

Now let's use the idea of adapting α as we go downhill.

Idea 1: More analysis. We choose α_n s.t it minimizes

$$\|s^{opt} - (s^n - \alpha_n \nabla \phi(s^n))\|.$$

$=: s^{n+1}$

Problems: • We don't know s^{opt} .

We can also show that, if ϕ is twice diff. and $v^T \phi'' v > 0$ for any v ,

then

$$\alpha_n = \frac{\|\nabla \phi(s^n)\|^2}{\nabla \phi(s^n)^T \phi'' \nabla \phi(s^n)} \quad \text{is an optimal learning rate.}$$

Idea 2: let's apply the formula:

$$\alpha_k = \left| \frac{(\nabla\phi(s^k) - \nabla\phi(s^{k-1})) \cdot (s^k - s^{k-1})}{\|\nabla\phi(s^k) - \nabla\phi(s^{k-1})\|^2} \right|$$

This will give larger steps in "flat regions" i.e. where $\nabla\phi$ does not change much. This is no extra effort since all quantities have already been computed.

Idea 3: Apply the formula: $\alpha_n = \frac{\alpha_0}{n+1}$ or $\alpha_n = \frac{\alpha_0}{\sqrt{n+1}}$.

Lecture 11

(Optional) Extensions of the gradient descent method

We have noticed that the gradient method is very simple, but contains some flaws. In particular, it is not so easy to find appropriate step sizes at each iteration

↳ should also check other directions at each iteration?
(not just $\nabla\phi(s^n)$)

↳ We can seek the minimum at all directions

General shape: $s^{n+1} = s^n - \alpha_n v^n$

↳ v^n is the search direction at step n .

Previously $v^n = \nabla\phi(s^n)$.

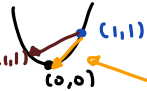
This is the best direction "locally", but it may not be ideal.

Example:

$$\phi(x, y) = 100x^2 + y^2$$

$$\bullet \text{ at } (1, 1): -\nabla\phi(1, 1) = \begin{pmatrix} -200 \\ -2 \end{pmatrix}$$

but we can see that if we take $(-1, -1)$, we arrive at the minimum in one step.



How can we choose v^n ?

let's consider ϕ a quadratic function: $\phi(s) = \frac{1}{2} s^T H s - b^T s + c$

Here H is symmetric positive definite

• We can show that $\phi''(s) = H$. That is, the second derivative of ϕ is a constant matrix.

- Geometrically, the graph of ϕ is something like: .

→ We can use this geometry and define a better dot product that is better fit for this problem:

$$(u, v)_H = u^T H v \rightarrow \text{measures the "angle" between } u, v \text{ using the Hessian matrix } H.$$

- Optimal directions**: we will choose directions that are conjugate with respect to $H = \phi''$.

↓

The conjugate gradient method

Some problems: Computing $H = \phi''$ can be too much work to find directions $v^n H v^k = 0$ ($k \neq n$).

If u and v are orthogonal in this sense, i.e.

$$u^T H v = 0$$

then we say they are conjugate. (with respect to H)

Remark: If $H = I$ (the identity matrix) then we have normal orthogonality: $u^T v = 0$

Instead, we find these directions iteratively.

Choosing optimal directions: the Fletcher-Reeves method

1. Start with a good initial guess s^0 and direction $d_0 = -\nabla \phi(s^0)$.

2. For $k = 0, 1, \dots, n-1$:

2.1. Minimize in direction d_k :

- Choose $\alpha_k \in \mathbb{R}$ such that $\phi(s - \alpha_k d_k)$ is minimal.

- Iterate $s^{k+1} = s^k - \alpha_k d_k$

This builds directions that are conjugate with respect to H

2.2. We construct a candidate for the new direction

$$g_{k+1} := \nabla \phi(s^{k+1})$$

2.3. If $k < n-2$, we correct this candidate to obtain a better new direction:

$$d_{k+1} = g_{k+1} + \underbrace{\frac{\|g_{k+1}\|^2}{\|g_k\|^2}}_{\text{correction term}} d_k$$

3. We finally have g_n and s^n .

4. If we accept our approximation, we finish. Otherwise we start again with $s^0 = s^n$ (and thus $d_0 = \nabla \phi(s^n)$).

Summarizing computational costs

In the context of fitting a model from some data, we have a minimizing function ψ and our model ϕ_a with parameters $a \in \mathbb{R}^M$. That is,

$$\sum_{k=1}^K \psi(\underbrace{\phi_a(x_k)}_{\text{model prediction}} - \underbrace{y_k}_{\text{output data}})$$

Input data.

e.g. Fitting data to a linear model, then $\phi_a(x) = a_1 x + a_2$, $a = (a_1, a_2) \in \mathbb{R}^2$.

At each step of the minimization process, we have to compute the gradient:

$$\left(\sum_{k=1}^K \psi(\phi_a(x_k) - y_k) \right)' = \sum_{k=1}^K [\psi(\phi_a(x_k) - y_k)]'$$

⚠ careful, we are differentiating with respect to the parameters $a = (a_1, a_2, \dots, a_M) \in \mathbb{R}^M$.

$$(\text{chain rule}) = \sum_{k=1}^K \psi'(\phi_a(x_k) - y_k) \cdot \nabla_a \phi_a(x_k)$$

Let's see how we can compute each term in the sum. Suppose:

$$\phi_a = f_N \circ f_{N-1} \circ \dots \circ f_1 \quad (N - \text{layers})$$

Then:

$$\psi'(\underbrace{f_N \circ \dots \circ f_1}_{\text{...}}(x)) \cdot f_N'(\underbrace{f_{N-1} \circ \dots \circ f_1}_{\text{...}}(x)) \cdot \dots \cdot f_1'(x)$$

If $f_i: \mathbb{R}^{d_{i-1}} \rightarrow \mathbb{R}^{d_i}$ (so the composition makes sense), then

$$f_i'(\dots) \in \mathbb{R}^{d_i \times d_{i-1}}$$

and $\psi: \mathbb{R}^{d_N} \rightarrow \mathbb{R}$, then $\psi'(\dots) \in \mathbb{R}^{1 \times d_N}$ [row vector so that multiplication makes sense]

Then: $\psi'(f_N \circ \dots \circ f_1(x)) \bullet f'_N(f_{N-1} \circ \dots \circ f_1(x))$

$$\begin{matrix} \downarrow & & \downarrow \\ (1, d_N) & (d_N, d_{N-1}) & = & (1, d_{N-1}) \end{matrix}$$

and each one of the d_{N-1} entries is a dot product $\Rightarrow d_{N-1} \cdot d_N$ multiplications total.

Next: $(1, d_{N-1}) \cdot (d_{N-1}, d_{N-2}) = (1, d_{N-2}) \Rightarrow d_{N-2} \cdot d_{N-1}$ multiplications total

$\vdots$
we have in total $d_N d_{N-1} + d_{N-1} d_{N-2} + \dots + d_1 d_0$ multiplications

Alternative: Compute the gradient symbolically. Then instead of these computations, the gradient method just requires evaluation of the gradient.

Remark: Number of operations until now depended roughly on the # of parameters.

Moreover, at each step of the process:

$$f_1 \rightarrow f_2 \rightarrow f_3 \rightarrow \dots \rightarrow f_N \quad (N: \text{number of layers})$$

we still need to perform these evaluations, per input x_k . If we have K total inputs, we must perform this K times.

Then, for a large number of inputs, this can be long.

However, some (input, output) pairs can be similar. This means that we could save computational costs by choosing only a certain # of (x, y) pairs.

$\downarrow$

Idea: Instead of $\sum_{k=1}^K [\psi(\phi_a(x_k) - y_k)]'$ in its entirety, we can take

$K_b \ll K$ (much smaller than K) and only consider the input-output pairs:

$$(x_1, y_1), (x_2, y_2), \dots, (x_{K_b}, y_{K_b}).$$

$$\Rightarrow \sum_{k=1}^{k_b} [\Psi(\phi_a(x_k) - y_k)]'$$

Then, we perform the gradient method based on this new sum:

$$a_1 = a_0 - \frac{\alpha}{k_b} \sum_{k=1}^{k_b} [\Psi(\phi_a(x_k) - y_k)]'$$

(1st iteration).

We repeat for $k = k_b+1, k_b+2, \dots, 2k_b$ and further until all input-output pairs are used:

$$\left. \begin{array}{l} \sum_{k=k_b+1}^{2k_b} [\dots]' ; \sum_{k=2k_b+1}^{3k_b} [\dots]' ; \dots \end{array} \right\} \begin{array}{l} a_2 = a_1 - \frac{\alpha}{k_b} \sum_{k=k_b+1}^{2k_b} [\dots]' \\ a_3 = a_2 - \frac{\alpha}{k_b} \sum_{k=2k_b+1}^{3k_b} [\dots]' \\ \vdots \end{array}$$

Terminology: One complete pass through our data set $(x_1, y_1), \dots, (x_k, y_k)$ is called an **epoch**. Each epoch will have a certain number of updates of the parameters a .

Remark: This is still a deterministic method but the approximation is usually a better approximation compared to a single conventional gradient step.

Taking $k_b \ll k$ is arbitrary. Furthermore, there is no need in choosing pairs in order. We can improve this by turning this into a **random process**.

↳

Principles of the stochastic gradient method.

We take a random sample of size $k_b \ll k$ from our data:

$$(x_{i_1}, y_{i_1}), (x_{i_2}, y_{i_2}), \dots, (x_{i_{k_b}}, y_{i_{k_b}})$$

This turns our sum (the steepest descent direction at some point in our iteration) into:

$$\sum_{k=1}^{k_b} [\Psi(\phi_a(x_{i_k}) - y_{i_k})]' \quad \text{for random indices } i_k \in \{1, \dots, n\}$$

- Then, we perform the gradient method as before.
- Repeat as long as necessary.

For a very small k_b , this update is cheap. However, this is at the cost of accuracy in the method.

Remark: For very small k_b , we can get at the end of the scheme relative large oscillations in the final loss function.

why? very different sampling may lead to different directions to move to during the gradient method

Example (see Jupyter notebook)

We have a real model $s(t) = \frac{1}{2}gt^2 + v_0 t$

$\left\{ \begin{array}{l} s: \text{distance of object.} \\ t: \text{time.} \\ v_0: \text{initial velocity object.} \\ g: \text{gravitational constant.} \end{array} \right.$

We generate data using this model with

$$g = 9.82; v_0 = 4$$

and then add noise to data to simulate measurements errors.

Goal: Use the gradient method to approximate g, v_0 .

what are we minimizing?

→ We want to find g, v_0 s.t

$$\| \phi_{g,v_0}(t) - y(t) \|_2^2 \text{ is minimal,}$$

where $y(t)$ is the observed data and $\phi_{g,v_0}(t) = s(t)$.

We will generate 3000 samples ($t = \text{np.linspace}(1, 4, 3000)$), i.e.

$$L(g, v_0) = \|\phi_{g, v_0}(t) - y(t)\|_2^2 = \sum_{i=1}^{3000} \underbrace{(\phi_{g, v_0}(t_i) - y(t_i))^2}_{\left(\frac{1}{2} g t_i^2 + v_0 t_i - y(t_i)\right)^2}$$

Then, the gradient is:

$$\begin{aligned} \nabla L(g, v_0) &= \sum_{i=1}^{3000} \left((\phi_{g, v_0}(t_i) - y(t_i))^2 \right)' \\ &= \sum_{i=1}^{3000} 2 (\phi_{g, v_0}(t_i) - y(t_i)) \begin{pmatrix} \partial_g \left(\frac{1}{2} g t_i^2 + v_0 t_i - y(t_i) \right) \\ \partial_{v_0} \left(\frac{1}{2} g t_i^2 + v_0 t_i - y(t_i) \right) \end{pmatrix} \\ &= \sum_{i=1}^{3000} 2 \left(\frac{1}{2} g t_i + v_0 t_i - y(t_i) \right) \begin{pmatrix} \frac{1}{2} t_i^2 \\ t_i \end{pmatrix} \end{aligned}$$

Then, the gradient method in this case is:

$$s^{n+1} = s^n - \alpha \nabla L(s^n)$$

and (hopefully) for small enough α , our iteration will converge to

$$s^* = \begin{pmatrix} 9.82 \\ 4 \end{pmatrix}.$$

Now, let's apply the **stochastic gradient method**. Take $k_b = 5 \ll 3000$.

That is, from $(\phi_{g, v_0}(t_i), y(t_i))$ we choose $i = i_1, i_2, i_3, i_4, i_5$ randomly

and compute one gradient step:

$$s^{n+1} = s^n - \alpha \nabla \bar{L}_i(s^n)$$

where

$$\bar{L}_i(\underbrace{g, v_0}_{s^n}) = \sum_{j=1}^5 (\phi_{g, v_0}(t_{ij}) - y(t_{ij}))^2$$

$$\Rightarrow \nabla \bar{L}_i(g, v_0) = \sum_{j=1}^5 2 \left(\frac{1}{2} g t_{ij} + v_0 t_{ij} - y(t_{ij}) \right) \begin{pmatrix} \frac{1}{2} t_{ij}^2 \\ t_{ij} \end{pmatrix}$$

In the code, we generate $\tilde{c}$ using `np.random.randint(0, 3000, size = 5)`,
i.e. from our index $0, 1, \dots, 2999$ we choose S randomly (our batch),
then update our parameter guess using the gradient method:

$$S = S - \alpha \nabla \bar{L}_i(S)$$

and repeat (1000 times in the show).